# Graph-Based ETL Pipeline Source Code

Complete codebase compilation including orchestration, extraction, transformation, loading, and quality checks.

## 1. extractor.py

```
import pandas as pd
import os

def extract_from_csv(file_path):
    # Check if the file exists before attempting to read
    if not os.path.exists(file_path):
        print(f"[ERROR] The data file {file_path} was not found.")
        raise FileNotFoundError(f"Missing file: {file_path}")

    print(f"[*] Starting extraction from {file_path}...")

    try:
        df = pd.read_csv(file_path)
        print(f"[+] Successfully extracted {df.shape[0]} rows and {df.shape[1]} columns.")
        return df
    except Exception as e:
        print(f"[ERROR] Failed to read the CSV file. Reason: {e}")
        raise
```

## 2. loader.py

```
import pandas as pd
import sqlite3

def load_to_database(df, table_name, db_name="graph_etl_warehouse.db"):
    print(f"\n[*] --- Loading data into table: {table_name} ---")

    try:
        # Establish connection to a local SQLite database
        conn = sqlite3.connect(db_name)

        # Load DataFrame into the database (replace table if it already exists)
        df.to_sql(table_name, conn, if_exists='replace', index=False)

        print(f"[+] Successfully loaded {len(df)} records into '{table_name}'.")

        # Close the connection
        conn.close()
    except Exception as e:
        print(f"[ERROR] Database load failed. Reason: {e}")
        raise
```

## 3. transformer.py

```
import pandas as pd
import numpy as np

def transform_data(df, dataset_name):
    # Create an explicit copy to avoid SettingWithCopyWarning
    df = df.copy()

    # Standardize column names
    df.columns = df.columns.str.strip().str.lower().str.replace(' ', '_')

    # Remove exact duplicate records
    initial_rows = len(df)
    df = df.drop_duplicates().copy()
    dropped = initial_rows - len(df)

    # Fill missing values for numerical columns using median
    numeric_cols = df.select_dtypes(include=[np.number]).columns
    for col in numeric_cols:
        if df[col].isnull().sum() > 0:
            median_val = df[col].median()
            df.loc[:, col] = df[col].fillna(median_val)

    # Fill missing values for string/object columns
    string_cols = df.select_dtypes(include=['object']).columns
    for col in string_cols:
        df.loc[:, col] = df[col].fillna("Unknown")
        df.loc[:, col] = df[col].astype(str).str.strip()

    print(f"    [+] Transformed {dataset_name}: {len(df)} valid rows ({dropped} duplicates
removed).")
    return df
```

## 4. quality_checks.py

```
import pandas as pd

def run_quality_checks(df, dataset_name):
    print(f"\n[*] --- Running Quality Checks for: {dataset_name} ---")
    is_valid = True

    # 1. Check if the dataframe is empty
    if df.empty:
        print(f"[ERROR] Validation Failed: {dataset_name} contains no data.")
        return False

    # 2. Check for duplicate IDs (if 'id' column exists)
    if 'id' in df.columns:
        duplicate_ids = df['id'].duplicated().sum()
        if duplicate_ids > 0:
            print(f"[ERROR] Validation Failed: Found {duplicate_ids} duplicate IDs.")
            is_valid = False

    # 3. Check for any remaining null values
```

## 5. test_transformer.py

```
import pandas as pd
import numpy as np

# Import the core transformation function
from transformer import transform_data

def test_full_transform_pipeline():
    """
    Test the data transformation logic to ensure it normalizes columns,
    removes duplicates, fills null values, and strips strings correctly.
    """

    # 1. Setup messy dummy data
    dummy_data = {
        "  Customer ID  ": [1, 2, 2, 3],
        " Product Name": [" Laptop ", "Mouse", "Mouse", np.nan],
        "Price ": [1000.0, 25.0, 25.0, np.nan]
    }
    df = pd.DataFrame(dummy_data)

    # 2. Run the transformation function
    clean_df = transform_data(df, "Test Dataset")

    # 3. Assertions (Validation Checks)

    # Check column normalization
    assert list(clean_df.columns) == ["customer_id", "product_name", "price"], \
        "Column names were not standardized correctly."

    # Check duplicate removal (Initial 4 rows should become 3 rows)
    assert len(clean_df) == 3, "Duplicate rows were not removed."

    # Check null value handling for numerics (Price median should replace NaN)
    assert not clean_df["price"].isnull().any(), "Null values remain in numeric columns."

    # Check string stripping and fillna
    assert clean_df["product_name"].iloc[0] == "Laptop", "String spaces were not stripped
properly."
    assert "Unknown" in clean_df["product_name"].values, "Null strings were not replaced
with 'Unknown'."

if __name__ == "__main__":
    try:
        test_full_transform_pipeline()
        print("\n[+] All transformation unit tests passed successfully!")
    except AssertionError as e:
        print(f"\n[ERROR] Test Failed: {e}")
```

## 6. app.py (Part 1)

```
import streamlit as st
import pandas as pd
import os
```

## 6. app.py (Part 2)

```
if st.sidebar.button("Run Pipeline & Benchmark"):
    with st.spinner("Generating synthetic datasets and running graph algorithms..."):
        generate_retail_datasets(data_volume, inject_fatal_error=inject_error)

        methods = [
            ("sequential", "Method A: Sequential Baseline"),
            ("topological", "Method B: Basic Topological"),
            ("proposed", "Method C: Proposed Critical-Path DAG")
        ]

        results = []
        task_logs = {}

        for mode_key, mode_label in methods:
            state = {}
            scheduler = GraphETLScheduler(max_workers=max_workers)
            build_retail_etl_dag(scheduler, state)
            res = scheduler.execute(mode=mode_key)

            results.append({
                "Scheduling Method": mode_label,
                "Makespan (Seconds)": round(res["makespan"], 4),
                "Completed Tasks": res["completed"],
                "Failed Tasks": res["failed"],
                "Blocked Tasks": res["blocked"]
            })
            if mode_key == "proposed":
                task_logs = res["task_metrics"]

        res_df = pd.DataFrame(results)

        # Calculate improvement
        seq_time = res_df.loc[0, "Makespan (Seconds)"]
        prop_time = res_df.loc[2, "Makespan (Seconds)"]
        improvement = round(((seq_time - prop_time) / seq_time) * 100, 2)

        # Display KPI Metrics
        col1, col2, col3, col4 = st.columns(4)
        col1.metric("Sequential Time", f"{seq_time} s")
        col2.metric("Proposed DAG Time", f"{prop_time} s")
        col3.metric("Speed Improvement", f"{improvement} %")
        col4.metric("Quality Gate Status", "BLOCKED" if inject_error else "PASSED")

        st.subheader("1. Performance Benchmark Results")
        st.dataframe(res_df, use_container_width=True)

        st.subheader("2. Execution Time Comparison")
        st.bar_chart(res_df.set_index("Scheduling Method")["Makespan (Seconds)"])

        st.subheader("3. Node-Level Execution Metrics (Proposed Method)")
        log_rows = [{"Task Node": k, "Status": v[0], "Duration (s)": round(v[1], 4)} for k,
v in task_logs.items()]
        st.table(pd.DataFrame(log_rows))
```

## 7. PaythonDraft_01.py (Part 1 - Data Generator)

```
import os
import time
import sqlite3
import numpy as np
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
from concurrent.futures import ThreadPoolExecutor, as_completed

from extractor import extract_from_csv
from transformer import transform_data
from quality_checks import run_quality_checks
from loader import load_to_database


# ==========================================================================
# 1. SYNTHETIC RETAIL DATA GENERATOR (WITH INTENTIONAL QUALITY ERRORS)
# ==========================================================================
def generate_retail_datasets(scale_rows, inject_fatal_error=False):
    os.makedirs("data", exist_ok=True)
    np.random.seed(42)

    n_cust = max(500, scale_rows // 10)
    n_prod = max(200, scale_rows // 25)
    n_store = 50
    n_sales = scale_rows

    # 1. Customers Dataset
    cust_df = pd.DataFrame({
        "ID": np.random.randint(1, n_cust + 1, size=n_cust),
        "Customer Name": [f"  Customer_{i} " if i % 15 != 0 else None for i in
range(n_cust)],
        "Loyalty Score": [np.random.uniform(10, 100) if i % 20 != 0 else np.nan for i in
range(n_cust)]
    }).drop_duplicates(subset=["ID"])
    cust_df.to_csv("data/customers.csv", index=False)

    # 2. Products Dataset
    prod_df = pd.DataFrame({
        "ID": np.arange(1, n_prod + 1),
        "Product Category": np.random.choice(["Electronics ", " Apparel", "Home", None],
size=n_prod),
        "Unit Price": np.random.uniform(5.0, 500.0, size=n_prod)
    })
    if inject_fatal_error:
        # Inject duplicate primary keys to trigger Data Quality Gate failure
        prod_df.loc[0:10, "ID"] = 999999
    prod_df.to_csv("data/products.csv", index=False)

    # 3. Stores Dataset
    store_df = pd.DataFrame({
        "ID": np.arange(1, n_store + 1),
        "Store Region": np.random.choice(["Colombo", "Kandy", "Galle", "Jaffna"],
size=n_store),
```

## 7. PaythonDraft_01.py (Part 2 - GraphETLScheduler Setup)

```
# ============================================================================
# 2. GRAPH-BASED ETL SCHEDULER (CRITICAL PATH & QUALITY GATE AWARE)
# ============================================================================
class GraphETLScheduler:
    def __init__(self, max_workers=3):
        self.graph = nx.DiGraph()
        self.max_workers = max_workers
        self.task_funcs = {}
        self.est_weights = {}
        self.critical_ranks = {}
        self.layers = {}

    def add_task(self, task_id, func, est_time, layer, dependencies=None):
        self.graph.add_node(task_id)
        self.task_funcs[task_id] = func
        self.est_weights[task_id] = est_time
        self.layers[task_id] = layer

        if dependencies:
            for dep in dependencies:
                self.graph.add_edge(dep, task_id)

    def compute_critical_path_ranks(self):
        """
        Calculates the upward rank (bottom-level critical path weight) for each node:
        Rank(u) = t_u + max_{v in succ(u)} Rank(v)
        """
        topo_order = list(nx.topological_sort(self.graph))
        for node in reversed(topo_order):
            successors = list(self.graph.successors(node))
            if not successors:
                self.critical_ranks[node] = self.est_weights[node]
            else:
                self.critical_ranks[node] = self.est_weights[node] + max(
                    self.critical_ranks[succ] for succ in successors
                )
        return self.critical_ranks

    def visualize_dag(self, save_path="etl_dag_architecture.png"):
        self.compute_critical_path_ranks()
        plt.figure(figsize=(15, 8))

        # Group nodes by layer for a clean left-to-right hierarchical layout
        layer_groups = {}
        for node, layer in self.layers.items():
            layer_groups.setdefault(layer, []).append(node)

        pos = {}
        for layer, nodes in layer_groups.items():
            y_positions = np.linspace(1.0, -1.0, len(nodes))
            for y, node in zip(y_positions, nodes):
                pos[node] = (layer * 2.5, y)

        color_map = []
```

## 7. PaythonDraft_01.py (Part 3 - Execution Engine)

```
    def execute(self, mode="proposed"):
        """
        Modes:
        - 'sequential': Method A (1 worker, basic topological order)
        - 'topological': Method B (Parallel workers, standard FIFO topological readiness)
        - 'proposed': Method C (Parallel workers, Critical-Path priority + Data Quality
Gate)
        """
        if not nx.is_directed_acyclic_graph(self.graph):
            raise ValueError("Pipeline graph contains a cycle! Execution aborted.")

        self.compute_critical_path_ranks()
        workers = 1 if mode == "sequential" else self.max_workers

        completed = set()
        failed = set()
        blocked = set()
        running = {}
        task_metrics = {}

        start_pipeline = time.perf_counter()

        with ThreadPoolExecutor(max_workers=workers) as executor:
            while len(completed) + len(failed) + len(blocked) < len(self.graph.nodes):
                # 1. Identify tasks whose upstream dependencies have failed/blocked
                for node in self.graph.nodes:
                    if node not in completed and node not in failed and node not in blocked
and node not in running:
                        preds = list(self.graph.predecessors(node))
                        if any(p in failed or p in blocked for p in preds):
                            blocked.add(node)
                            print(f"    [QUALITY GATE BLOCK] Downstream task '{node}'
blocked due to upstream failure.")

                # 2. Find ready tasks
                ready_tasks = [
                    node for node in self.graph.nodes
                    if node not in completed and node not in failed and node not in blocked
and node not in running
                    and all(p in completed for p in self.graph.predecessors(node))
                ]

                # 3. Prioritize tasks based on scheduling algorithm
                if mode == "proposed":
                    ready_tasks.sort(key=lambda n: self.critical_ranks[n], reverse=True)

                # 4. Dispatch tasks to available worker threads
                while ready_tasks and len(running) < workers:
                    next_task = ready_tasks.pop(0)
                    t_start = time.perf_counter()
                    future = executor.submit(self.task_funcs[next_task])
                    running[future] = (next_task, t_start)

                # 5. Collect completed futures
```

## 7. PaythonDraft_01.py (Part 4 - DAG Construction)

```
# ======================================================================
# 3. BUILD THE 14-NODE RETAIL ETL PIPELINE DAG
# ======================================================================
def build_retail_etl_dag(scheduler, state):
    # Simulated I/O and processing complexity weights (in seconds)
    def run_step(delay, fn):
        time.sleep(delay)
        return fn()

    # Layer 0: Parallel Extractions
    scheduler.add_task("Ext_Cust", lambda: run_step(0.15, lambda: state.update({"cust_raw":
extract_from_csv("data/customers.csv")})), 0.15, 0)
    scheduler.add_task("Ext_Prod", lambda: run_step(0.12, lambda: state.update({"prod_raw":
extract_from_csv("data/products.csv")})), 0.12, 0)
    scheduler.add_task("Ext_Store", lambda: run_step(0.08, lambda:
state.update({"store_raw": extract_from_csv("data/stores.csv")})), 0.08, 0)
    scheduler.add_task("Ext_Sales", lambda: run_step(0.30, lambda:
state.update({"sales_raw": extract_from_csv("data/sales.csv")})), 0.30, 0)

    # Layer 1: Parallel Transformations
    scheduler.add_task("Trf_Cust", lambda: run_step(0.18, lambda:
state.update({"cust_clean": transform_data(state["cust_raw"], "Customers")})), 0.18, 1,
["Ext_Cust"])
    scheduler.add_task("Trf_Prod", lambda: run_step(0.14, lambda:
state.update({"prod_clean": transform_data(state["prod_raw"], "Products")})), 0.14, 1,
["Ext_Prod"])
    scheduler.add_task("Trf_Store", lambda: run_step(0.10, lambda:
state.update({"store_clean": transform_data(state["store_raw"], "Stores")})), 0.10, 1,
["Ext_Store"])
    scheduler.add_task("Trf_Sales", lambda: run_step(0.35, lambda:
state.update({"sales_clean": transform_data(state["sales_raw"], "Sales")})), 0.35, 1,
["Ext_Sales"])

    # Layer 2: Data Quality Validation Gates
    def make_validator(key, name):
        def _val():
            time.sleep(0.10)
            if not run_quality_checks(state[key], name):
                raise RuntimeError(f"Quality Gate failed on {name}")
        return _val

    scheduler.add_task("Val_Cust", make_validator("cust_clean", "Customers"), 0.10, 2,
["Trf_Cust"])
    scheduler.add_task("Val_Prod", make_validator("prod_clean", "Products"), 0.10, 2,
["Trf_Prod"])
    scheduler.add_task("Val_Store", make_validator("store_clean", "Stores"), 0.10, 2,
["Trf_Store"])
    scheduler.add_task("Val_Sales", make_validator("sales_clean", "Sales"), 0.15, 2,
["Trf_Sales"])

    # Layer 3: Star-Schema Relational Join / Enrichment
    def join_warehouse_tables():
        time.sleep(0.25)
        sales = state["sales_clean"]
```

## 7. PaythonDraft_01.py (Part 5 - Experimental Benchmarks)

```
# ======================================================================
# 4. EXPERIMENTAL BENCHMARK SUITE & PERFORMANCE CHARTS
# ======================================================================
def run_experimental_benchmarks():
    volumes = [10000, 50000, 100000]
    methods = [
        ("sequential", "Method A: Sequential Baseline"),
        ("topological", "Method B: Basic Topological"),
        ("proposed", "Method C: Proposed Critical-Path DAG")
    ]

    results_list = []

    print("\n" + "=" * 70)
    print(" RUNNING EXPERIMENTAL EVALUATION ACROSS DATA VOLUMES & METHODS")
    print("=" * 70)

    for rows in volumes:
        print(f"\n>>> Generating Synthetic Retail Dataset ({rows:,} Sales Rows)...")
        generate_retail_datasets(rows, inject_fatal_error=False)

        for mode_key, mode_label in methods:
            print(f"\n--- Running {mode_label} ({rows:,} rows) ---")
            state = {}
            scheduler = GraphETLScheduler(max_workers=3)
            build_retail_etl_dag(scheduler, state)

            res = scheduler.execute(mode=mode_key)
            results_list.append({
                "Dataset_Rows": rows,
                "Method": mode_label,
                "Mode_Key": mode_key,
                "Makespan_Sec": round(res["makespan"], 4),
                "Completed_Tasks": res["completed"],
                "Blocked_Tasks": res["blocked"]
            })
            print(f"==> Finished {mode_label} | Makespan: {res['makespan']:.3f}s")

    results_df = pd.DataFrame(results_list)

    # Calculate Improvement (%) of Proposed Method C over Method A & Method B
    print("\n" + "=" * 70)
    print(" EXPERIMENTAL RESULTS SUMMARY TABLE")
    print("=" * 70)
    pivot_df = results_df.pivot(index="Dataset_Rows", columns="Method",
values="Makespan_Sec")
    pivot_df["Improvement_vs_Seq (%)"] = (
        (pivot_df["Method A: Sequential Baseline"] - pivot_df["Method C: Proposed Critical-
Path DAG"])
        / pivot_df["Method A: Sequential Baseline"] * 100
    ).round(2)
    pivot_df["Improvement_vs_Topo (%)"] = (
        (pivot_df["Method B: Basic Topological"] - pivot_df["Method C: Proposed Critical-
Path DAG"])
```

## 7. PaythonDraft_01.py (Part 6 - Plotting & Main)

```
    # Plot Performance Comparison Chart
    plt.figure(figsize=(10, 6))
    x = np.arange(len(volumes))
    width = 0.25

    m_a = results_df[results_df["Mode_Key"] == "sequential"]["Makespan_Sec"].values
    m_b = results_df[results_df["Mode_Key"] == "topological"]["Makespan_Sec"].values
    m_c = results_df[results_df["Mode_Key"] == "proposed"]["Makespan_Sec"].values

    plt.bar(x - width, m_a, width, label="Method A: Sequential Baseline", color="#EF5350")
    plt.bar(x, m_b, width, label="Method B: Basic Topological (Parallel)", color="#42A5F5")
    plt.bar(x + width, m_c, width, label="Method C: Proposed Critical-Path DAG",
color="#66BB6A")

    plt.xlabel("Dataset Volume (Number of Sales Records)", fontsize=11, fontweight="bold")
    plt.ylabel("Total Execution Time / Makespan (Seconds)", fontsize=11, fontweight="bold")
    plt.title("ETL Pipeline Scheduling Performance Comparison", fontsize=13,
fontweight="bold")
    plt.xticks(x, [f"{v:,} Rows" for v in volumes])
    plt.legend()
    plt.grid(axis="y", linestyle="--", alpha=0.6)
    plt.tight_layout()
    plt.savefig("performance_comparison_chart.png", dpi=300)
    print("[+] Saved performance comparison chart to 'performance_comparison_chart.png'")
    plt.show()

    # Demonstrate Data Quality Gate Failure Isolation
    print("\n" + "=" * 70)
    print(" TESTING DATA QUALITY VALIDATION GATE (FAULT ISOLATION DEMO)")
    print("=" * 70)
    generate_retail_datasets(10000, inject_fatal_error=True)
    fault_state = {}
    fault_scheduler = GraphETLScheduler(max_workers=3)
    build_retail_etl_dag(fault_scheduler, fault_state)
    fault_res = fault_scheduler.execute(mode="proposed")
    print(
        f"==> Fault Test Result: {fault_res['completed']} Tasks Succeeded | "
        f"{fault_res['failed']} Failed (Val_Prod) | "
        f"{fault_res['blocked']} Downstream Tasks Blocked (Join_DW, Load_DW)!"
    )


if __name__ == "__main__":
    # 1. Build & Visualize the 14-Node DAG Architecture
    demo_state = {}
    demo_scheduler = GraphETLScheduler(max_workers=3)
    build_retail_etl_dag(demo_scheduler, demo_state)
    print("[*] Visualizing 14-Node Multi-Source ETL Pipeline DAG...")
    demo_scheduler.visualize_dag()

    # 2. Run Full Experimental Benchmarks (10k, 50k, 100k rows)
    run_experimental_benchmarks()
```